FINDIALOGLENS: A NOVEL HYBRID LLM FRAMEWORK FOR PARSING FINANCIAL DIALOGUES AND UNVEILING MISSED TRADE OPPORTUNITIES

Anonymous authors
Paper under double-blind review

ABSTRACT

Unstructured communication channels, such as chatrooms and instant messaging platforms, are essential for financial professionals to share ideas, negotiate trades, discuss pricing, and understand client needs. These dynamic, multi-party conversations are rich in trading opportunities and market insights, but their volume and complexity make manual analysis impractical and error-prone. Extracting actionable trade outcomes from multi-line, multi-threaded dialogues among traders, sales teams, and clients remains a significant challenge, often resulting in missed opportunities and incomplete records. To address this gap, we present **FinDialogLens**, a novel framework that systematically analyzes financial dialogues and identifies missed trading opportunities. FinDialogLens integrates three specialized modules: (1) Message-Level Module, (2) RFQ-Level Module, and (3) Trade Engine. The Message-Level Module leverages compact, domain-adapted, fine-tuned models to provide precise financial knowledge, such as RFQ identification, trade outcome classification, and price extraction, for each message. This metadata is then used by the RFQ-Level Module to construct targeted Request for Quote (RFQ) queries, enabling the Trade Engine to accurately extract final prices and trade outcomes. Our results show that FinDialogLens integrating with GPT-4o Trade Engine substantially improves extraction accuracy, achieving rates of 0.921 for final price and 0.943 for trade outcome. In comparison, Chain-of-Thought (CoT) prompting with GPT-4o achieves only 0.609 and 0.664 in the zero-shot setting, and 0.644 and 0.708 in the few-shot setting. Furthermore, we demonstrate that fine-tuned open-source LLMs, such as Mistral-7B-Instruct, can match the performance of proprietary models like GPT-4o with a modest amount of in-domain data. Finally, we introduce a dynamic routing module that optimizes the balance between performance and cost, reducing LLM API calls by 85% while maintaining high accuracy, making FinDialogLens suitable for deployment in budget-constrained environments.

1 INTRODUCTION

Unstructured communication channels have been widely adopted across the financial industry to facilitate idea sharing, conduct research, inquire about trades, discuss pricing, and more. The real-time and historical conversations within these dialogue systems serve as an invaluable treasure trove, offering the potential to discover new trading opportunities and enabling the automatic capture of trade parameters in trade structuring, derivative pricing, and meeting pre-trade reporting requirements for regulatory compliance applications. By systematically analyzing these interactions, financial analysts and traders can gain valuable insights into market trends, client preferences, and emerging opportunities that might otherwise remain untapped.

While executed trades are consistently recorded, missed trades often remain unlogged due to the cumbersome and time-consuming nature of the current manual process. This oversight can result in gaps in understanding client needs or potential trades that could have been secured. One such analysis that can be extremely valuable would be identifying missed trades in these chats. This enables financial analysts and traders to refine strategies and improve decision-making by uncovering strategy shortcomings and discovering new market opportunities. A typical trade conversation usu-

ally begins with a message from one party that includes the price inquiry, Request for Quote (RFQ), followed by pricing from the other party, price negotiation between both parties, and finally, a trade outcome (hit, miss, or unknown) to indicate whether the trade occurred or not.

One potential approach to address challenges outlined above is to implement dialogue disentanglement (Li et al., 2024) on financial dialogues. The goal of dialogue disentanglement is to separate chronologically ordered utterances into distinct, independent sessions. Previous studies on dialogue disentanglement (Mehri & Carenini, 2017; Liu et al., 2020; Huang et al., 2022) typically employed classifiers to identify pair-wise *reply-to* relations based on the contextual features of dialogues. However, it is not well suited to our problem. Firstly, existing research indicates that training classifiers for dialogue disentanglement requires a substantial amount of labeled data, which involves a labor-intensive annotation process. For instance, considerable effort was needed to prepare the Ubuntu IRC data (Kummerfeld et al., 2019). Secondly, despite the significant effort involved, even the current State-of-the-Art methods exhibit a performance gap relative to our requirements. Lastly, in our specific context, not all messages hold equal importance. In financial dialogues, messages containing RFQs, prices, and trade outcomes are significantly more critical than others.

To effectively identify and prioritize important messages, our approach employs three modules: (1) Message-Level Module, (2) RFQ-Level Module, and (3) Trade Engine, as illustrated in Figure 1. The Message-Level Module consists of several compact models that provide domain-specific knowledge for each message, including RFQ identification, trade outcome classification, and price entity extraction. The resulting metadata are then utilized by the RFQ-Level Module to construct contextual RFQ queries, which enable the Trade Engine to accurately extract final prices and trade outcomes.

We benchmarked our approach against the Chain-of-Thought (CoT) prompting method (Wei et al., 2022) as a baseline. As demonstrated in Section 3.2.1, incorporating financial knowledge from the Message-Level Module and reducing input complexity through RFQ chunking in the RFQ-Level Module significantly improves extraction performance. FinDialogLens, when integrated with GPT-4o, achieves accuracy rates of 0.921 for final price and 0.943 for trade outcome. Notably, fine-tuning open-source LLMs with a modest amount of training data enables them to approach proprietary model performance: FinDialogLens with fine-tuned Mistral-7B-Instruct (Jiang et al., 2023) attains 0.890 and 0.936 accuracy for final price and trade outcome, respectively. In contrast, CoT prompting with GPT-4o yields only 0.609 and 0.664 accuracy in the zero-shot setting, and 0.644 and 0.708 in the few-shot setting.

The landscape of LLMs is evolving rapidly, with both proprietary and open-source options such as GPT-4 (Achiam et al., 2023), Mistral, and Llama-3 (Dubey et al., 2024) offering varying performance and cost profiles. As these options expand, organizations face a trade-off between performance and cost: more powerful LLMs deliver superior results but at higher expense, while smaller LLMs are more cost-effective but may underperform in complex scenarios. To address this, researchers have proposed routing approaches to reduce costs while maintaining performance (Chen et al., 2023; Aggarwal et al., 2024). However, these methods often increase latency by querying multiple LLMs. In contrast, the work of Ding et al. (2024) and Ong et al. (2025) routes each query to only one system (i.e., a strong or a weak system) during inference, reducing latency and computational overhead.

Building on this idea, we introduce the Routing Module to dynamically route RFQ queries between two systems. Specifically, we have developed RuleTrade, which leverages conversation characteristics and domain-specific metadata from the Message-Level Module to identify final prices and trade outcomes for each RFQ using a set of predefined rules. While effective, RuleTrade faces challenges in handling complex scenarios, such as cases where multiple RFQs occur within a short time frame. In Section 3.2.3, we demonstrate how our Routing Module balances performance and cost by directing RFQ queries between RuleTrade and FinDialogLens (GPT-4o), which utilizes the GPT-4o Trade Engine. This approach offers practical benefits: as LLMs advance, existing systems can be incrementally enhanced with LLM solutions without requiring complete replacement, thus mitigating engineering challenges and costs. In budget-constrained settings, the Routing Module optimizes resource allocation by allowing more powerful LLMs to focus on complex tasks, maintaining high performance without significant compromise.

To summarize, we make the following contributions:

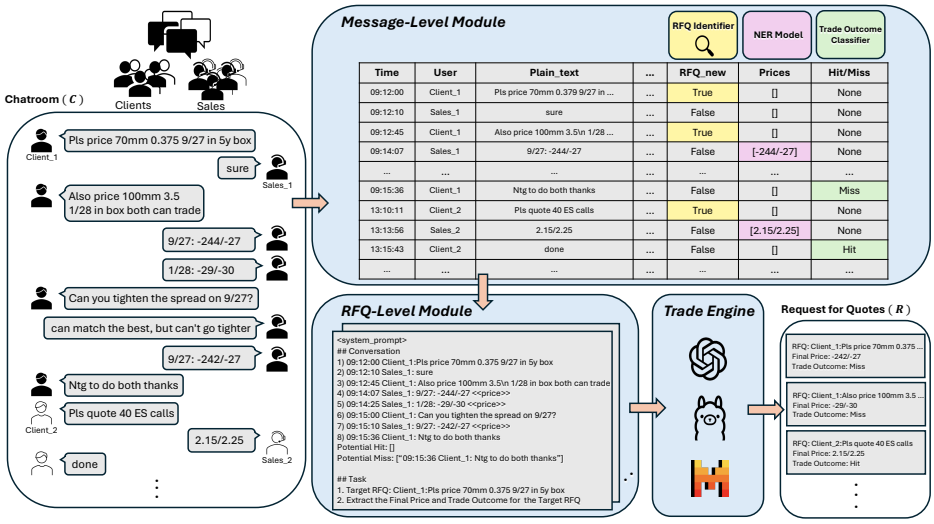


Figure 1: Overview of the proposed FinDialogLens framework for extracting RFQs (R) from financial dialogues (C). The framework initiates with a Message-Level Module, comprising an RFQ Identifier, Trade Outcome Classifier, and a Named Entity Recognition (NER) Model, which collectively provide domain-specific insights for each message. Leveraging this information, the RFQ-Level Module constructs contextual RFQ queries, enabling the Trade Engine to accurately extract final prices and trade outcomes.

- To our knowledge, FinDialogLens is the first framework to systematically address the complex problem of missed trade identification in financial dialogues by integrating three specialized modules: the Message-Level Module, RFQ-Level Module, and Trade Engine. The framework is designed to be flexible and extensible, supporting future integration of new models and methods.
- FinDialogLens significantly outperforms LLM Chain-of-Thought prompting, achieving over 25% higher accuracy in both final price and trade outcome by generating more fine-grained RFQ queries through the Message-Level and RFQ-Level Modules. The framework is adaptable to various small open-source LLMs as the Trade Engine, and we demonstrate that fine-tuning these models with a modest amount of training data is both feasible and scalable.
- We introduce a Routing Module that optimizes resource allocation by directing RFQ queries between RuleTrade and FinDialogLens (GPT-4o). Leveraging a RoBERTa-based model as the router enables daily cost savings of over $300 while maintaining comparable performance, with the potential for even greater savings as usage increases. This approach ensures cost-efficient performance improvements and facilitates seamless integration of different LLMs within the pipeline.

2 METHODOLOGY

We present our framework, FinDialogLens, which consists of three components: (a) Message-Level Module, (b) RFQ-Level Module, and (c) Trade Engine. Additionally, we present the Routing Module, which is designed to optimize the integration and performance of two systems: RuleTrade and FinDialogLens (GPT-4o), while efficiently managing LLM API costs.

2.1 PROBLEM SETTING

Given a chatroom $C = \{m_i\}_{i=1}^{N}$ containing N messages, the objective of our framework is to identify the set of RFQs $R = \{r_j\}_{j=1}^{K}$, where K denotes the number of RFQs present in C. For

each RFQ $r_j \in R$, we aim to extract the corresponding RFQ message $m_j \in C$, its final price p_j, and its trade outcome o_j.

Formally, each RFQ is represented as:

$$r_j = \{m_j, p_j, o_j\}, \tag{1}$$

where m_j is the RFQ message, p_j is the final price, and o_j is the trade outcome associated with r_j.

2.2 Message-Level Module

Inspired by recent work (Yu et al., 2023) demonstrating that fine-tuning compact models such as RoBERTa with sufficient data can outperform larger models like GPT-4 in specialized tasks. The Message-Level Module is designed to extract key financial insights from each message in the conversation using a set of fine-tuned RoBERTa-based models. Specifically, we employ three specialized models: an RFQ Identifier to detect Request for Quote (RFQ) messages, a Trade Outcome Classifier to determine whether a trade was executed ("Hit") or missed ("Miss"), and a Named Entity Recognition (NER) Model to extract price-related entities. The RFQ Identifier and Trade Outcome Classifier are formulated as traditional classification tasks, while the NER model is trained as a standard named entity recognition task. For each message $m_i \in C$, these models generate structured outputs that are stored as metadata and subsequently used by downstream modules in our framework.[1]

2.3 RFQ-Level Module

A typical daily chatroom C contains over 400 messages and more than 20 RFQs. Within a single chat, multiple clients and sales representatives may negotiate prices for different products, and messages frequently include financial jargon and abbreviations. This complexity poses significant challenges for LLMs attempting to directly extract the final price and trade outcome for each RFQ from the entire conversation.

To address this, the RFQ-level Module leverages metadata generated by the Message-Level Module to construct targeted RFQ queries, allowing LLMs to focus on relevant segments of the conversation. Formally, we define a set of targeted RFQ queries $Q = \{q_j\}_{j=1}^{K}$, where K is the number of messages identified as RFQs by the RFQ Identifier. The construction process is as follows:

1. Use the message m_j identified the RFQ Identifier as the starting message for the query q_j.
2. Collect subsequent messages $\{m_{j+1}, m_{j+2}, \ldots\} \subset C$ until either a 20-minute time limit or a 25-message count limit is reached from the starting message m_j.[2]
3. Integrate additional message-level metadata, including outputs from the Trade Outcome Classifier and NER Model, into the query q_j.[3]

2.4 Trade Engine

Given each targeted RFQ query $q_j \in Q$, the Trade Engine aims to extract the final price p_j and trade outcome o_j for the target RFQ message m_j. The output is a tuple (m_j, p_j, o_j), representing the target RFQ message, its extracted final price, and trade outcome. While a single query may contain multiple RFQ messages, the Trade Engine focuses exclusively on extracting information for m_j, defined as the first message in the query q_j. Both RuleTrade and LLM-powered engines leverage message-level metadata to enhance extraction accuracy.

2.4.1 RuleTrade

RuleTrade utilizes key characteristics of financial dialogues to construct a set of predefined, straightforward yet effective rules that associate potential prices and trade outcomes that are identified by the Message-Level Module with the appropriate RFQs. The predefined rules are based on features

[1] Further details of the Message-Level Module are provided in Appendix A.2.1.

[2] Limits are determined based on statistical analysis of typical RFQ conversations. A 20-minute time limit or a 25-message count limit is able to cover over 96% RFQs.

[3] The way of integrating these message-level metadata is shown in Figure 8.

such as reply structure, product references, response behavior, and temporal proximity. Despite its simplicity and static nature, RuleTrade effectively captures RFQs that adhere to typical dialogue patterns, particularly in cases where RFQs are clearly separated from each other. However, real-world financial dialogues often involve multiple overlapping RFQs, ambiguous references, and multi-participant threads, making it difficult for static rules to accurately locate the prices and trade outcomes for those RFQs.

2.4.2 LLM-POWERED ENGINE

We leverage the natural language understanding capabilities of LLMs to associate final prices and trade outcomes with targeted RFQ messages. We explore two types of LLMs:

1. Proprietary LLMs: We perform prompt engineering on GPT-series models using the evaluation dataset and select the most effective prompt for the test dataset.
2. Open-source LLMs (e.g., Mistral): We transform the training dataset using a specific instruction template and fine-tune the models through instruction-tuning (Zhang et al., 2023).

Throughout this paper, FinDialogLens (GPT-4o) refers to our framework using GPT-4o as the Trade Engine, and similar variants indicate the use of the respective LLM as the Trade Engine.

2.5 ROUTING MODULE

In practical applications, budget constraints and latency requirements are critical considerations. To address these challenges, we introduce a routing module, denoted as $Router$, designed to optimize the efficiency of LLM API usage within FinDialogLens while maintaining high performance. Specifically, this work explores the dynamic allocation of RFQ queries between a strong engine, E_{strong}, and a weak engine, E_{weak}. In this study, E_{strong} refers to FinDialogLens (GPT-4o), whereas E_{weak} corresponds to RuleTrade, which operates without LLM API calls. The Routing Module determines the optimal assignment of each RFQ query to either engine as follows:

$$Router : Q \rightarrow \{E_{\text{strong}}, E_{\text{weak}}\} \tag{2}$$

We collect training data $D_{\text{train}} = \{(q, l) \mid q \in Q, l \in L\}$, where l indicates the correctness of RuleTrade for query q, with $L = \{\text{Correct}, \text{Wrong}\}$. We fine-tune the routers for query classification, which outputs the probability of RuleTrade's correctness $P_\theta(\text{Correct} \mid q)$, where θ represents the router parameters. We optimize θ by maximizing the likelihood of the training data:

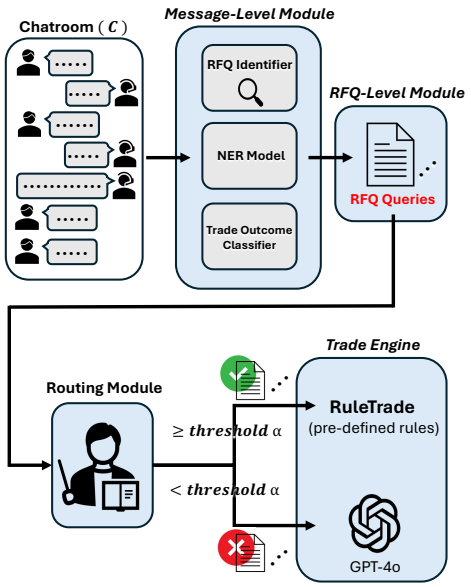


Figure 2: Overview of the Routing Module for dynamically allocating RFQ queries between FinDialogLens (GPT-4o) and RuleTrade.

$$\max_{\theta} \sum_{(q,l) \in D_{\text{train}}} \log P_\theta(l \mid q) \tag{3}$$

A cost threshold α is used to balance quality and cost in the routing function $Router^\alpha$. Given an input query q, the routing decision is made as follows:

$$Router^\alpha(q) = \begin{cases} E_{\text{weak}}, & \text{if } P(\text{Correct} \mid q) \geq \alpha \\ E_{\text{strong}}, & \text{otherwise} \end{cases} \tag{4}$$

In this paper, we explore three types of fine-tuned routers, including an encoder-only classifier (i.e., RoBERTa-base Liu et al. (2019)), a decoder-only classifier (i.e., Mistral-7B-Instruct), and an encoder-decoder classifier (i.e., Flan-T5-XL Chung et al. (2024)).

3 EXPERIMENTS

In this section, we provide a comprehensive overview of the data used, annotation guidelines, training details, metrics, and the results achieved.

3.1 EXPERIMENT SETUP

3.1.1 DATA OVERVIEW

We annotated 47,362 messages, including 8,343 RFQs, from two years of financial conversations (2023 and 2024). The dataset is split into non-overlapping training, evaluation, and test subsets. The training set is used to fine-tune RoBERTa models for the Message-Level Module and open-source LLMs for the Trade Engine and Routing Module. The evaluation set supports model selection and prompt engineering, while all reported results are based on the held-out test set. Given the sensitive nature of financial communications, we enforce strict privacy protocols to prevent any risk of data leakage or unauthorized access. Details of data statistics, privacy protocols, and annotation guidelines are provided in the Appendix A.1.

3.1.2 BASELINES

As our baseline, we apply the Chain-of-Thought (CoT) prompting method using GPT-4o on daily chat data, with the "gpt-4o-2024-05-13" API version. CoT prompting is evaluated under four settings: (1) GPT-4o zero-shot, (2) GPT-4o few-shot, (3) GPT-4o zero-shot with domain adaptation, and (4) GPT-4o few-shot with domain adaptation. For domain adaptation, message-level metadata is incorporated into the daily chat context.[4] In the few-shot setting, we provide three examples. Detailed prompt template for each setting is provided in Figure 5.

3.1.3 FINDIALOGLENS

To comprehensively evaluate our proposed framework, we detail the architecture and training of each core component within FinDialogLens, including the Message-Level Module, Trade Engine, and Routing Module.

The Message-Level Module comprises three models: the RFQ Identifier, Trade Outcome Classifier, and Named Entity Recognition (NER) model. Each model is based on RoBERTa-base and is fine-tuned on the training dataset, achieving F1 scores of 0.97, 0.95, and 0.93, respectively, on the test set.[5] For the Trade Engine, we employ a unified prompt template—illustrated in Figure 6—across all engines, including GPT-4o ("gpt-4o-2024-05-13" API version), Mistral-7B-Instruct, Llama-3-8B-Instruct, Phi-3-mini-4k-instruct, and Flan-T5-XL. The training data are reformatted using this prompt template, and the open-source LLMs are fine-tuned using Low-Rank Adaptation (LoRA) (Hu et al., 2022) with a rank of 64, alpha of 16, and a dropout rate of 0.1 applied to all attention layers. For the Routing Module, RoBERTa-base is fully fine-tuned as the router, while Mistral-7B-Instruct and Flan-T5-XL are fine-tuned using LoRA (rank 64, alpha 16, dropout 0.1 on all attention layers). The prompt template for both Mistral-7B-Instruct and Flan-T5-XL is shown in Figure 7. All models are trained on a machine equipped with 4 NVIDIA A10G GPUs. Further training details, including the number of epochs, batch size, and learning rates, are provided in Appendix A.2.

3.1.4 METRICS

We report both accuracy and F1 metrics for final price and trade outcome. The exact match for final price requires the extracted value to be identical to the price as it appears in the chat, such as "50/53" or "14.78 mid," without any normalization or post-processing. For trade outcome, the label

[4] An example of incorporating message-level metadata into daily chat is shown in Figure 8.

[5] The detailed performance of the Message-Level Module is presented in Table 5.

Table 1: Comparison of final price and trade outcome extraction performance across different methods. The "w/ Domain Adapt." column indicates whether the input context is augmented with message-level metadata from the Message-Level Module. The "Input Context" column specifies the type of conversation provided to the Trade Engine. Note that all open-source LLMs are fine-tuned using the entire training dataset.

Method	w/ Domain Adapt.	Input Context		Trade Engine	**Final Price**		**Trade Outcome**	
		Daily Chat	RFQ Queries		Acc	F1	Acc	F1
RuleTrade	✓		✓	Pre-defined Rules	77.1	86.5	90.3	94.9
CoT Prompting		✓		GPT-4o (Zero-shot)	60.9	75.5	66.4	75.9
		✓		GPT-4o (Few-shot)	64.4	78.1	70.8	80.0
	✓	✓		GPT-4o (Zero-shot)	73.9	84.6	85.4	92.7
	✓	✓		GPT-4o (Few-shot)	79.2	88.1	86.7	93.5
FinDialogLens	✓		✓	GPT-4o	**92.1**	**95.9**	**94.3**	**97.1**
	✓		✓	Mistral-7B (Fine-tuned)	89.0	93.9	93.6	96.7
	✓		✓	Llama-3-8B (Fine-tuned)	86.3	92.4	91.0	95.3
	✓		✓	Phi-3-mini-4k (Fine-tuned)	87.4	93.0	89.9	94.7
	✓		✓	Flan-T5-XL (Fine-tuned)	89.1	94.0	93.4	96.6

is considered an exact match if it is one of "Hit," "Miss," or "Unknown," as assigned in the annotation process. Here, "Hit" indicates that the trade was executed, "Miss" indicates that the trade was not executed, and "Unknown" refers to cases where the client did not respond to the sales inquiry. To evaluate the performance of the Routing Module, we follow the methodology of RouterLLM (Ong et al., 2025), employing the Performance Gap Recovery (PGR) and Call-Performance Threshold (CPT) metrics. The PGR is defined as follows:

$$PGR(Router^{\alpha}) = \frac{r(Router^{\alpha}) - r(E_{\text{weak}})}{r(E_{\text{strong}}) - r(E_{\text{weak}})},$$

where E_{strong} and E_{weak} are FinDialogLens (GPT-4o) and RuleTrade, respectively, and $r(.)$ represents response quality, measured by accuracy. CPT(x%) measures the minimum percentage of calls to GPT-4o within FinDialogLens needed to achieve the specified PGR of x%.

3.2 Results

In this section, we present a comprehensive evaluation of FinDialogLens, highlighting three key aspects: (1) the enhancement of LLM performance for RFQ extraction through the integration of financial knowledge from compact models and the construction of targeted RFQ queries; (2) the generalizability of our approach across open-source LLMs of varying sizes and architectures; and (3) the effectiveness of the Routing Module in reducing LLM API calls and associated costs by intelligently directing RFQ queries between RuleTrade and FinDialogLens (GPT-4o).

3.2.1 Domain Adaptation and Structured Querying on LLM Performance

Table 1 presents a comprehensive comparison of our proposed framework, FinDialogLens, against baseline methods including RuleTrade and various CoT prompting strategies. The "w/ Domain Adapt." column indicates whether message-level metadata, comprising RFQ identification, trade outcome classification, and price-related entity extraction from the Message-Level Module, was incorporated into the input context. As illustrated in Figure 8, these metadata elements are integrated into the conversation as special tokens, providing the LLM with domain-specific cues. For CoT prompting baselines, GPT-4o is prompted using daily chat dialogue, while RuleTrade and FinDialogLens leverage contextual RFQ queries generated by the RFQ-Level Module. This distinction in input context is reflected in the "Input Context" columns of the table.

Our results demonstrate that augmenting GPT-4o with domain-specific message-level metadata yields substantial performance gains. Specifically, in the zero-shot setting, domain adaptation im-

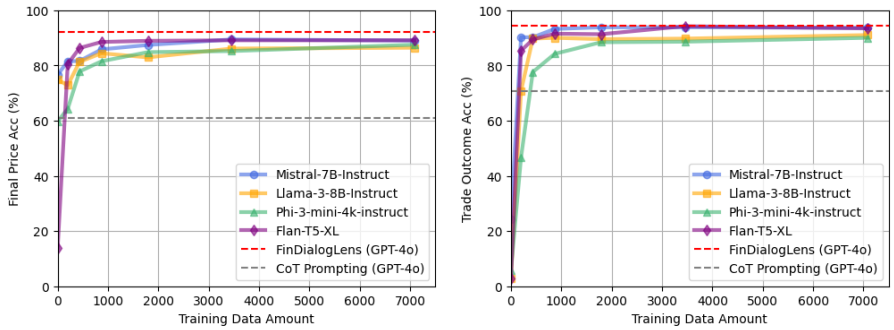


Figure 3: Performance comparison of open-source LLMs in extracting final prices and trade outcomes across varying amounts of training data. The x-axis represents the amount of training data, and the y-axis represents the accuracy for final prices and trade outcomes. The red dashed horizontal line indicates the accuracy of FinDialogLens (GPT-4o) while the gray dashed horizontal line indicates the accuracy of GPT-4o with CoT prompting in zero-shot setting.

proves final price and trade outcome accuracy by 13.0% and 19.0%, respectively. In the few-shot setting, the improvements are 14.8% and 15.9%. These findings highlight the value of integrating compact, fine-tuned models to provide financial knowledge and context for LLMs in complex dialogue scenarios. Furthermore, the introduction of the RFQ-Level Module in FinDialogLens significantly reduces the complexity of financial dialogue extraction. By constructing targeted RFQ queries, FinDialogLens with GPT-4o achieves an additional 18.2% and 8.9% accuracy improvement over CoT prompting with domain adaptation in the zero-shot setting, and 12.9% and 7.6% in the few-shot setting, for final price and trade outcome, respectively. Notably, our fine-tuned Mistral-7B-Instruct model also outperforms CoT prompting with domain adaptation in the few-shot setting, achieving 9.8% higher accuracy for final price and 6.9% higher accuracy for trade outcome.

In summary, the Message-Level Module provides essential domain-specific information that enhances LLM-based RFQ extraction, while the RFQ-Level Module further boosts performance by enabling more precise and context-aware querying. These results underscore the effectiveness of our proposed framework in advancing the state of financial dialogue understanding.

3.2.2 Open-source Vs. Proprietary LLMs

Figure 3 presents the performance trajectories of fine-tuned open-source LLMs within the FinDialogLens framework as the amount of training data increases. We observe that model accuracy improves rapidly with additional data, eventually reaching a plateau where further increases in training size (beyond approximately 1,000 examples) yield diminishing returns. At this saturation point, the performance of open-source LLMs closely approaches that of GPT-4o within FinDialogLens.

Notably, all four fine-tuned open-source LLMs trained on the entire dataset outperform GPT-4o with Chain-of-Thought (CoT) prompting baselines, as shown in Table 1. For instance, fine-tuned Flan-T5-XL achieves accuracy scores of 0.891 for final price and 0.934 for trade outcome, whereas CoT prompting with GPT-4o in a few-shot setting without domain adaptation yields only 0.644 and 0.708, respectively. These results highlight the effectiveness of our framework and demonstrate that, with a modest amount of labeled data, open-source LLMs with as few as 3 billion parameters can match or even surpass the performance of much larger proprietary models. Furthermore, the model-agnostic design of FinDialogLens enables seamless integration of various open-source LLMs, offering both flexibility and cost-efficiency for real-world deployment.

3.2.3 Router: Balance Performance and Cost

Table 2 reports the Call-Performance Thresholds (CPT) for four routing strategies. Lower CPT values indicate greater efficiency, as fewer queries must be routed to GPT-4o to achieve a target performance. Fine-tuned routers, such as Flan-T5-XL and RoBERTa-base, require substantially fewer GPT-4o calls than the Random baseline to reach the same accuracy. For example, Flan-T5-

XL achieves 80% PGR in final price accuracy with only 29.9% of queries sent to GPT-4o, compared to 81.3% for the Random router. Similar efficiency gains are observed for trade outcome metrics. Figure 4 further confirms that fine-tuned routers maintain higher accuracy in final price as the proportion of GPT-4o calls decreases (i.e., as the threshold α is varied). Among the fine-tuned models, performance is comparable, with no single model consistently dominating.

Table 3 presents the cost analysis for processing around 1,000 RFQs. Processing 1,000 queries exclusively with GPT-4o costs $5.4, whereas the RoBERTa-based router reduces the cost to just $0.62 under CPT(50%), corresponding to approximately 85% final price accuracy. At a scale of 70,000 RFQs per day, this results in daily savings exceeding $300, without sacrificing accuracy.[6] In summary, fine-tuned routers substantially reduce LLM API costs while maintaining extraction accuracy, making our proposed framework both scalable and cost-effective for real-world deployment.

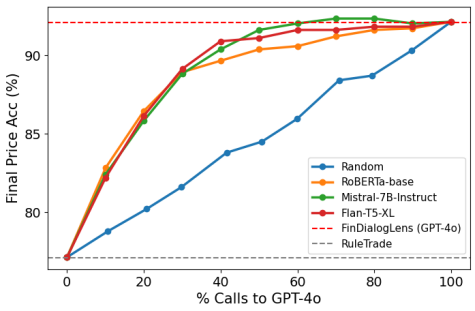


Figure 4: Performance of routers under various budget constraints. The red dashed horizontal line indicates the accuracy of FinDialogLens (GPT-4o), while the gray dashed horizontal line indicates the accuracy of RuleTrade.

Table 2: Comparison of Call-Performance Threshold (CPT) for different router types on final price and trade outcome. Lower CPT indicates greater efficiency.

Router Type	Final Price		Trade Outcome	
	CPT (50%)	CPT (80%)	CPT (50%)	CPT (80%)
Random	49.2%	81.3%	52.1%	75.8%
RoBERTa-base	**14.9%**	32.4%	29.5%	62.6%
Mistral-7B	16.4%	31.8%	26.7%	36.3%
Flan-T5-XL	16.1%	**29.9%**	**25.1%**	**32.8%**

Table 3: Cost analysis of different router types for final price and trade outcome under CPT (50%) and CPT (80%). Reported values indicate the cost per 1,000 RFQs.

Router Type	Final Price		Trade Outcome	
	CPT (50%)	CPT (80%)	CPT (50%)	CPT (80%)
Random	$2.66	$4.39	$2.81	$4.09
RoBERTa-base	**$0.80**	$1.75	$1.59	$3.38
Mistral-7B	$0.89	$1.72	$1.44	$1.96
Flan-T5-XL	$0.87	**$1.61**	**$1.36**	**$1.77**

4 Conclusion

We present FinDialogLens, a novel framework for automating the extraction of RFQs, final prices, and trade outcomes from unstructured communication channels. By integrating compact, domain-adapted models for message-level analysis, targeted RFQ-level querying, and a flexible Trade Engine, FinDialogLens systematically addresses the challenge of missed trade identification in complex financial dialogues. Our results show that FinDialogLens, when combined with GPT-4o or fine-tuned open-source LLMs using a modest amount of labeled data, surpasses the CoT prompting baselines in both final price and trade outcome extraction. The model-agnostic design of FinDialogLens allows seamless integration of various LLMs, ensuring flexibility and scalability. Additionally, the dynamic routing module optimizes the balance between performance and cost, reducing daily expenses by over $300 without sacrificing accuracy. These features make FinDialogLens a practical and cost-effective solution for real-world deployment in budget-constrained environments.

5 Limitations

While effective, our approach has some limitations. The collection of subsequent messages after an RFQ is based on fixed time and message count thresholds, which may not capture all edge cases. Additionally, our current routing mechanism only supports two systems, whereas real-world scenarios may require more flexible, multi-system routing strategies. Ideally, a fine-tuned router should dynamically assess performance and cost across multiple systems for optimal query allocation.

[6] We report the average of tokens and cost per RFQ and the latency analysis in Appendix A.3.

6 Ethics Statement

This research is driven by the goal of enhancing operational efficiency in financial markets through the automated analysis of unstructured communication channels, such as chatrooms and instant messaging platforms. The proposed framework, FinDialogLens, is designed to support financial professionals in identifying missed trading opportunities and improving decision-making, while complementing, rather than replacing, human expertise and judgment.

The financial data and conversational content analyzed in this study are highly sensitive and managed in accordance with strict industry governance and privacy protocols. All experiments were conducted using anonymized, internally governed datasets, with user identities replaced by generic identifiers to ensure privacy. No personal or client-identifying information was collected, stored, or utilized at any stage. Access to both data and model outputs is strictly restricted to authorized personnel, and all data handling complies with relevant regulatory and organizational standards.

The sample dialogues presented in this paper do not represent the opinions or positions of the authors or their organizations. We are committed to maintaining the highest standards of data privacy, security, and research integrity throughout this work.

7 Reproducibility Statement

Due to the confidential nature of the financial data used in this study, public release of the raw data is not possible. However, to support reproducibility, we provide comprehensive data statistics, annotation guidelines, and training details in the Appendix. All model architectures, hyperparameters, and evaluation protocols are described in detail in the main text and Appendix.

References

Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

Pranjal Aggarwal, Aman Madaan, Ankit Anand, Srividya Pranavi Potharaju, Swaroop Mishra, Pei Zhou, Aditya Gupta, Dheeraj Rajagopal, Karthik Kappaganthu, Yiming Yang, et al. Automix: Automatically mixing language models. *Advances in Neural Information Processing Systems*, 37:131000–131034, 2024.

Lingjiao Chen, Matei Zaharia, and James Zou. Frugalgpt: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.

Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, et al. Scaling instruction-finetuned language models. *Journal of Machine Learning Research*, 25(70):1–53, 2024.

Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Ruhle, Laks VS Lakshmanan, and Ahmed Hassan Awadallah. Hybrid llm: Cost-efficient and quality-aware query routing. *arXiv preprint arXiv:2404.14618*, 2024.

Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models, 2024.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *ICLR*, 1(2):3, 2022.

Chengyu Huang, Zheng Zhang, Hao Fei, and Lizi Liao. Conversation disentanglement with bi-level contrastive learning. *arXiv preprint arXiv:2210.15265*, 2022.

Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de Las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas

Wang, Timothée Lacroix, and William El Sayed. Mistral 7b. *CoRR*, abs/2310.06825, 2023. doi: 10.48550/ARXIV.2310.06825.

Jonathan K. Kummerfeld, Sai R. Gouravajhala, Joseph J. Peper, Vignesh Athreya, Chulaka Gunasekara, Jatin Ganhotra, Siva Sankalp Patel, Lazaros C Polymenakos, and Walter Lasecki. A large-scale corpus for conversation disentanglement. In Anna Korhonen, David Traum, and Lluís Màrquez (eds.), *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 3846–3856, Florence, Italy, July 2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1374.

Bobo Li, Hao Fei, Fei Li, Shengqiong Wu, Lizi Liao, Yinwei Wei, Tat-seng Chua, and Donghong Ji. Revisiting conversation discourse for dialogue disentanglement. *ACM Trans. Inf. Syst.*, 43(1), November 2024. ISSN 1046-8188. doi: 10.1145/3698191.

Hui Liu, Zhan Shi, Jia-Chen Gu, Quan Liu, Si Wei, and Xiaodan Zhu. End-to-end transition-based online dialogue disentanglement. In *IJCAI*, volume 20, pp. 3868–3874, 2020.

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.

Shikib Mehri and Giuseppe Carenini. Chat disentanglement: Identifying semantic reply relationships with random forests and recurrent neural networks. In *Proceedings of the Eighth International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pp. 615–623, 2017.

Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs from preference data. In *The Thirteenth International Conference on Learning Representations*, 2025.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.

Hao Yu, Zachary Yang, Kellin Pelrine, Jean Francois Godbout, and Reihaneh Rabbany. Open, closed, or small language models for text classification? *arXiv preprint arXiv:2308.10092*, 2023.

Shengyu Zhang, Linfeng Dong, Xiaoya Li, Sen Zhang, Xiaofei Sun, Shuhe Wang, Jiwei Li, Runyi Hu, Tianwei Zhang, Fei Wu, et al. Instruction tuning for large language models: A survey. *arXiv preprint arXiv:2308.10792*, 2023.

A Appendix

Table 4: Statistics of human-labeled data.

Dataset	#message	#RFQ
Training Dataset	37245	7082
Evaluation Dataset	1147	295
Test Dataset	8970	966

Table 5: Performance of the Message-Level Module, including the RFQ Identifier, Trade Outcome Classifier, and Named Entity Recognition (NER) Model.

Model	Precision	Recall	F1
RFQ Identifier	0.97	0.97	0.97
Trade Outcome Classifier	0.96	0.93	0.95
NER Model (price-related)	0.91	0.95	0.93

A.1 Data Privacy, Annotation Guidelines, and Statistics

Privacy: To ensure the privacy and confidentiality of conversations used in our framework, we implemented dedicated anonymization measures. Specifically, user names within the conversations are replaced with generic identifiers such as "Client_" and "Sales_," effectively safeguarding individual identities.

Annotation Guidelines: To guarantee the accuracy and reliability of our human-labeled data, we adopted a rigorous annotation protocol involving two professional annotators who independently reviewed each message and each Request for Quote (RFQ). For message-level annotation, both annotators labeled the RFQ message, trade outcome classification, and price information for each message. For RFQ-level annotation, each annotator independently assigned both the final price and the trading outcome, categorized as Hit, Miss, or Unknown, for every RFQ. Here, "Hit" indicates that the trade was executed, "Miss" indicates that the trade was not executed, and "Unknown" refers to cases where the client did not respond to the sales inquiry. Only data points with full consensus between annotators were retained.

Statistics: The human-labeled data comprises a total of 47,362 annotated messages, including 8,343 RFQs, collected from two years of financial conversations (2023 and 2024). As shown in Table 4, the data is partitioned into three non-overlapping subsets: 37,245 messages (7,082 RFQs) for training, 1,147 messages (295 RFQs) for evaluation, and 8,970 messages (966 RFQs) for testing.

A.2 Training Details of FinDialogLens

All experiments are conducted with a fixed random seed of 42 and, where applicable, a temperature setting of 0. Model training is performed on a machine equipped with 4 NVIDIA A10G GPUs.

A.2.1 Message-Level Module

Inspired by recent work (Yu et al., 2023) demonstrating that fine-tuning compact models such as RoBERTa with sufficient data can outperform larger models like GPT-4 in specialized tasks, we fine-tune three RoBERTa-base models within the Message-Level module to generate more sophisticated financial outputs for each message. Formally, for each message $m_i \in C$:

- **RFQ Identifier**: Determines whether a message m_i constitutes a Request for Quote (RFQ). We define a binary function:

$$y_i^{\text{RFQ}} = f_{\text{RFQ}}(m_i) \in \{True, False\} \tag{5}$$

  where $y_i^{\text{RFQ}} = True$ indicates that m_i is identified as an RFQ message.
- **Trade Outcome Classifier**: Determines whether a message m_i indicates a trade outcome as either "Hit", "Miss", or "None." We define:

$$y_i^{\text{trade}} = f_{\text{trade}}(m_i) \in \{\text{Hit}, \text{Miss}, \text{None}\} \tag{6}$$

- **NER Model**: Identifies and extracts the price entity from the message content. We define:

$$y_i^{\text{price}} = f_{\text{NER}}(m_i) \tag{7}$$

  where y_i^{price} is the list of extracted price values from m_i.

The predictions y_i^{RFQ}, y_i^{trade}, and y_i^{price} are stored as metadata for each message m_i and are utilized in the subsequent module of our framework. The precision, recall, and F1 scores for these models are reported in Table 5.

A.2.2 Trade Engine

For GPT-4o engine, we use "gpt-4o-2024-05-13" API version. We fine-tune four open-source LLMs as Trade Engines: (1) Mistral-7B-Instruct, (2) Llama-3-8B-Instruct, (3) Phi-3-mini-4k-instruct, and (4) Flan-T5-XL. The unified prompt template for all LLMs is shown in Figure 6. The training configuration is as follows:

1. Batch_size = 2
2. Gradient_accumulation_steps = 16
3. LoRAConfig = (lora_alpha = 16, lora_dropout = 0.1, r = 64)
4. Optimizer: AdamW

5. Epochs and learning rates (vary by training dataset size):

#Train	#Epoch	Learning rate
200	10	5e-5
435	10	5e-5
870	10	5e-5
1790	20	5e-5
3455	20	5e-5
7082	20	5e-5

A.2.3 ROUTING MODULE

In this section, we elaborate on the training details for three types of routers:

a) **Encoder-only Classifier**: RoBERTa-base is fully fine-tuned for this classifier. A classification layer is applied to the <s> token embedding to determine the class label L. The model is trained using cross-entropy loss, optimizing the parameters to accurately classify the input queries based on the provided labels.

b) **Decoder-only Classifier**: Mistral-7B-Instruct is fine-tuned using a prompt template to generate a single token, either "Yes" or "No." A "Yes" indicates that the input query is correct, while a "No" indicates incorrectness. The probability of correctness is computed using a softmax over "Yes" and "No," with "Yes" corresponding to $P_\theta(\text{Correct} \mid q)$.

c) **Encoder-Decoder Classifier**: Flan-T5-XL is fine-tuned for this classifier, employing a training and inference pipeline similar to that of the Decoder-only Classifier. The model leverages both encoder and decoder components to process input queries and generate a single token, either "Yes" or "No". A "Yes" indicates that the input query is correct, while a "No" indicates incorrectness.

Below are the training configurations:

RoBERTa-base:

1. Epoch = 5
2. Learning_rate = 2e-5
3. Batch_size = 32
4. Gradient_accumulation_steps = 8
5. Optimizer: AdamW

Mistral-7B-Instruct and Flan-T5-XL:

1. Prompt template is shown in Figure 7
2. Epoch = 5
3. Learning_rate = 1e-4
4. Batch_size = 4
5. Gradient_accumulation_steps = 32
6. LoRAConfig = (lora_alpha = 16, lora_dropout = 0.1, r = 64)
7. Optimizer: AdamW

Table 6: Latency analysis of different router types for final price and trade outcome under CPT(50%) and CPT(80%). Reported values indicate the LLM API latency time in second for processing all test dataset. The total LLM API latency is 1459 second with processing all test dataset with GPT-4o.

Router Type	Final Price		Trade Outcome	
	CPT (50%)	CPT (80%)	CPT (50%)	CPT (80%)
Random	717.8	1186.2	760.1	1105.9
RoBERTa-base	**217.4**	427.7	430.4	913.3
Mistral-7B	239.3	464.0	389.6	529.6
Flan-T5-XL	234.9	**436.2**	**366.2**	**478.6**

A.3 Cost and Latency Analysis

We provide a detailed analysis of cost and latency for the routing experiments. The average cost per RFQ using GPT-4o is $0.0054, with an average of 1,024.4 input tokens and 18.5 output tokens per RFQ. As shown in Table 3, processing over 70,000 RFQs per day with the RoBERTa-base router results in daily savings exceeding $300, while maintaining approximately 85% accuracy for final price extraction.

For latency, the average response time for a single GPT-4o API call is 1.51 seconds per RFQ. In our latency experiments, we measured the total processing time by sequentially submitting one query at a time, without batching or parallelization. The total latency for processing the entire test dataset with GPT-4o is approximately 1,459 seconds.

Table 6 summarizes the total LLM API latency (in seconds) for different router types under CPT(50%) and CPT(80%) thresholds. The results show that fine-tuned routers, such as RoBERTa-base and Flan-T5-XL, substantially reduce overall latency compared to the Random baseline. For example, under CPT(50%), the RoBERTa-base router reduces the total latency for final price extraction to 217.4 seconds, compared to 717.8 seconds for the Random router. Similarly, Flan-T5-XL achieves a latency of 234.9 seconds under CPT(50%) for final price, and 366.2 seconds for trade outcome, both significantly lower than the Random router's 760.1 seconds for trade outcome. These reductions highlight the efficiency and practicality of our framework in real-world deployment.

A.4 LLM Usage Disclosure

In accordance with ICLR 2026 requirements, we disclose the use of Large Language Models (LLMs) in the preparation of this paper:

- **Writing Assistance:** LLMs (such as GPT-4) were used to polish the language and improve the clarity of sentences throughout the manuscript. All scientific content, experimental design, and analysis were authored by the paper's contributors.
- **Literature Search:** LLMs were used to assist in searching for related works and summarizing relevant literature. All cited works were verified and selected by the authors.

No content generated by LLMs was used as original scientific contribution, and all results, experiments, and analyses were performed and interpreted by the authors. The authors take full responsibility for the accuracy and integrity of the manuscript.

You are a smart financial assistant designed to provide final prices, and trade outcomes for all RFQs (Request for Quote) appeared in the conversation.

Background Knowledge
[Provide a brief introduction to RFQs, including what an RFQ is and how clients and sales teams negotiate prices.]

Tips
[Include step-by-step Chain-of-Thought guidelines for RFQ extraction.]

Conversation
{**Input Content**}

Special tokens *(Include this section only when message-level metadata is present in the Input Content)*
[Describe the meaning of each special token, e.g., <<rfq>>, <<hit>>, <<miss>>, and <<price>>.]

Examples *(Include this section only for the few-shot setting)*
[Insert few-shot examples here.]

Task
[Output Format: describe the output formant]

Figure 5: Prompt template for CoT prompting methods.

You are a smart financial assistant designed to provide final price, and trade outcome for the targeted RFQ (Request for Quote) appeared in the conversation.

Background Knowledge
[Provide a brief introduction to RFQs, including what an RFQ is and how clients and sales teams negotiate prices.]

Tips
[Include step-by-step Chain-of-Thought guidelines for RFQ extraction.]

Conversation
{**Input Content**}

Special tokens
[Describe the meaning of each special token, e.g., <<rfq>>, <<hit>>, <<miss>>, and <<price>>.]

Task
[Output Format: describe the output formant]

Figure 6: Prompt template for the Trade Engine within FinDialogLens.

You are a helpful evaluator. Given a conversation, your task is to determine whether the rule-based system can successfully detect the final price and trading outcome for the target RFQ.

Background Knowledge
[Provide a brief introduction to RFQs, including what an RFQ is and how clients and sales teams negotiate prices.]

Rule-based System
[Provide the pre-defined rules used in the rule-based system.]

Conversation
{**Input Content**}

Task
[Output Format: describe the output formant]

Figure 7: Prompt template for the Routing Module within FinDialogLens.

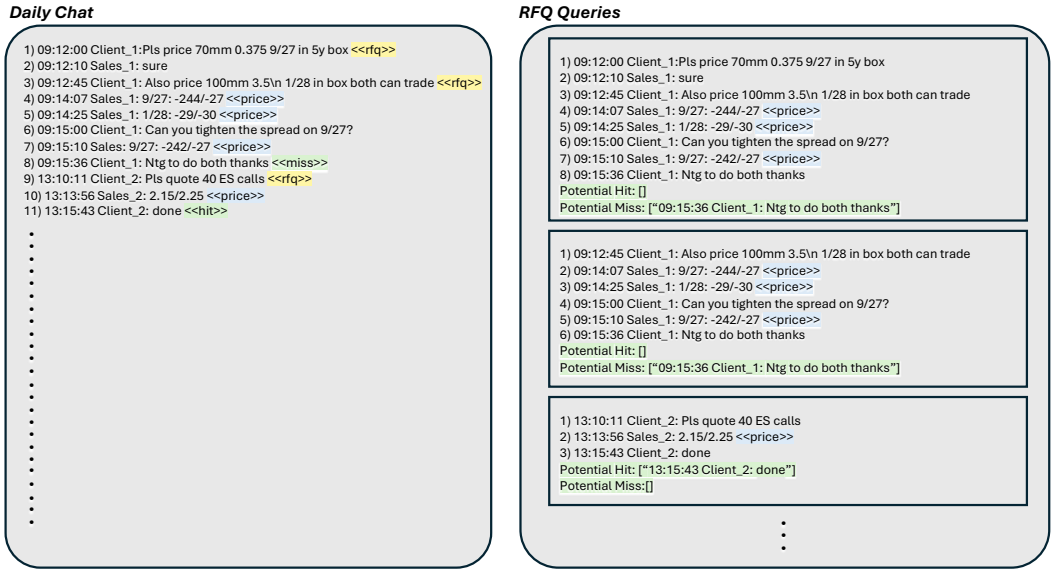


Figure 8: Examples of the two types of **Input Context**: Daily Chat and RFQ Queries, used in the prompt template. Message-level metadata from the Message-Level Module is incorporated into the dialogue using special tokens, such as ≪rfq≫, ≪hit≫, ≪miss≫, and ≪price≫. For CoT prompting methods, LLMs are prompted with the full daily chat, whereas our proposed FinDialogLens framework prompts LLMs with structured RFQ queries.